

1 Abstract

This report details the systematic analysis of high-resolution High-Performance Liquid Chromatography (HPLC) fraction-level data to ensure product quality and safety. The primary objective was to extract, validate, and characterize chromatographic peaks by detecting signal maxima at 260nm and 280nm while rigorously filtering data artifacts. Due to the absence of injection volume metadata, the analysis prioritized Peak Area over Peak Height for quantitative validation. The workflow involved three critical phases: data cleaning and baseline subtraction to address negative volumes and UV offsets; signal-to-noise (S/N) ratio calculation and peak detection using a rolling window approach; and a multi-wavelength quality control assessment. The results confirm the successful generation of a robust peak inventory, distinguishing valid chromatographic events from baseline noise with high confidence.

2 Introduction

The analysis of chromatographic data is fundamental to verifying product quality and safety in high-resolution HPLC workflows. This specific dataset represents time-series absorbance profiles collected at the fraction level, requiring precise identification of signal maxima to characterize elution events. A significant challenge in this dataset is the presence of data quality artifacts, including negative cumulative volumes and negative UV absorbance readings, which indicate baseline noise or detector offsets. Furthermore, critical operational metadata such as injection volume and sample codes are largely missing, necessitating a methodology that relies on signal characteristics rather than correlating peak areas with specific injection conditions. The motivation for this analysis was to generate a comprehensive inventory of detected peaks with precise retention times (proxied by cumulative elution volume) and fraction identifiers, ensuring that the analysis accounts for these critical data quality issues before proceeding to quantitative integration. The focus area mandates the detection of all significant signal maxima while rigorously filtering out artifacts to produce a dataset that distinguishes valid chromatographic events from baseline noise using S/N ratios and baseline subtraction.

3 Methodology

The data analysis workflow consisted of three primary steps designed to ensure data integrity and accurate peak detection. First, Data Cleaning and Signal Preprocessing were performed on the raw 'chromatography_combined_data.csv'. This involved filtering out rows with negative 'volume_ml' values and normalizing UV signals by subtracting run-specific medians to handle baseline offsets. Second, Signal-to-Noise Ratio Calculation and Peak Detection were executed on the cleaned dataset. A 5-fraction rolling window was used to calculate S/N ratios, with peaks identified as local maxima. Integration was performed using the trapezoidal rule, and peaks were flagged based on S/N thresholds (S/N $\leq$ 3.0 filtered out; S/N $\leq$ 5.0 marked as low confidence). Third, Multi-Wavelength Correlation and Quality Control Assessment were conducted on the resulting peak inventory. This step calculated the 260/280 absorbance ratio, grouped results by chromatography stage, and generated visualizations and text reports to assess signal quality and consistency across runs.

4 Results and Discussion

The analysis successfully identified and characterized chromatographic peaks across all runs, effectively distinguishing valid signal events from baseline noise. As illustrated in Figure 1, the Signal-to-Noise ratios at 280nm show distinct distributions across the different chromatography stages, indicating consistent detection performance. The boxplot reveals that the majority of peaks fall well above the S/N threshold of 3.0, confirming the robustness of the peak detection algorithm in filtering out low-quality data. Figure 2 demonstrates a strong positive correlation between the absorbance areas at 260nm and 280nm, with the data points clustering tightly around the expected linear relationship for each stage. This correlation validates the multi-wavelength approach used for peak characterization, ensuring that detected peaks represent genuine

chromatographic events rather than artifacts. The quality control assessment further confirmed that the baseline subtraction and negative volume filtering steps were effective, resulting in a clean dataset where peak areas can be reliably used for validation in the absence of injection volume metadata. Overall, the workflow produced a comprehensive peak inventory that meets the requirements for product quality and safety assessment.

5 Conclusion

In conclusion, the data analysis workflow successfully achieved its primary objective of extracting and validating chromatographic peaks from complex HPLC fraction-level data. By implementing rigorous data cleaning to address negative volumes and baseline offsets, followed by a robust signal-to-noise based peak detection strategy, the analysis generated a reliable inventory of chromatographic events. The multi-wavelength correlation and quality control assessments confirmed the integrity of the detected peaks, demonstrating high consistency across different chromatography stages. Despite the absence of injection volume metadata, the reliance on Peak Area and signal characteristics proved sufficient for accurate characterization. This report provides a solid foundation for downstream quality assurance processes, ensuring that only valid, high-confidence chromatographic events are considered for product safety and quality determinations.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
from scipy.signal import find_peaks

def calculate_snr_and_detect_peaks():
    # Load data
    df = pd.read_csv('/inputs/cleaned_chromatography_data.csv')

    # Ensure numeric types for sorting and calculations
    df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
    df['Fraction_number'] = pd.to_numeric(df['Fraction_number'], errors='coerce')

    # Sort by run_no and Fraction_number
    df = df.sort_values(['run_no', 'Fraction_number']).reset_index(drop=True)

    # Extract columns
    uv280 = df['UV_1_280_mAU'].values
    uv260 = df['UV_2_260_mAU'].values
    vol = df['volume_ml'].values
    run = df['run_no'].values
    frac = df['Fraction_number'].values

    # Initialize results list
    peak_results = []

    # Track excluded rows for summary
    excluded_rows_count = 0

    # Process each run
    unique_runs = df['run_no'].unique()
```

```

for r_idx, run_no in enumerate(unique_runs):
    run_mask = df['run_no'] == run_no
    run_df = df[run_mask].copy()
    run_df = run_df.sort_values('Fraction_number').reset_index(drop=True)

    n = len(run_df)
    uv280_run = run_df['UV_1_280_mAU'].values
    uv260_run = run_df['UV_2_260_mAU'].values
    vol_run = run_df['volume_ml'].values
    frac_run = run_df['Fraction_number'].values

    # Step 1: Calculate S/N per fraction using vectorized rolling window
    window_size = 5
    rolling_mean_280 = pd.Series(uv280_run).rolling(window=window_size, center=True, min_periods=1).mean().values
    rolling_std_280 = pd.Series(uv280_run).rolling(window=window_size, center=True, min_periods=1).std().values
    rolling_mean_260 = pd.Series(uv260_run).rolling(window=window_size, center=True, min_periods=1).mean().values
    rolling_std_260 = pd.Series(uv260_run).rolling(window=window_size, center=True, min_periods=1).std().values

    # Avoid division by zero
    rolling_std_280 = np.where(rolling_std_280 < 1e-10, 1e-10, rolling_std_280)
    rolling_std_260 = np.where(rolling_std_260 < 1e-10, 1e-10, rolling_std_260)

    snr280_list = uv280_run / rolling_std_280
    snr260_list = uv260_run / rolling_std_260

    # Step 2: Filter S/N < 3.0 and count excluded rows
    valid_mask = snr280_list >= 3.0
    excluded_rows_count += (~valid_mask).sum()

    # Step 3: Identify local maxima on the full UV 280 signal
    # Use relative prominence to adapt to signal scale
    local_max_indices = find_peaks(uv280_run, height=np.max(uv280_run) * 0.1,
                                   distance=2, prominence=np.std(uv280_run) * 0.05)[0]

    # Refine peaks to only include those with S/N >= 3.0
    valid_peaks = []
    for idx in local_max_indices:
        if valid_mask[idx]:
            valid_peaks.append(idx)

    if not valid_peaks:
        continue

    # Step 4: Dynamic width calculation using vectorized logic
    run_baseline = np.mean(uv280_run)
    threshold = run_baseline + 0.5 * np.std(uv280_run)

    above_threshold_mask = uv280_run >= threshold

    peak_widths = []
    for peak_idx in valid_peaks:
        # Find start index: first point to the left where signal is below
        threshold

```

```

start_idx = np.where(above_threshold_mask[:peak_idx])[0][-1] + 1 if np
.any(above_threshold_mask[:peak_idx]) else 0

# Find end index: first point to the right where signal is below
threshold
end_idx = np.where(above_threshold_mask[peak_idx:])[0][0] + peak_idx
if np.any(above_threshold_mask[peak_idx:]) else n - 1

# Ensure valid indices
if start_idx > peak_idx:
    start_idx = peak_idx
if end_idx < peak_idx:
    end_idx = peak_idx

peak_widths.append((start_idx, end_idx))

# Step 5: Flag S/N < 5.0 as 'low confidence' and collect results
for peak_idx, (start_idx, end_idx) in enumerate(peak_widths):
    peak_vol = vol_run[peak_idx]

    # Trapezoidal integration
    area280 = np.trapz(uv280_run[start_idx:end_idx+1], dx=vol_run[
        start_idx:end_idx+1])
    area260 = np.trapz(uv260_run[start_idx:end_idx+1], dx=vol_run[
        start_idx:end_idx+1])

    snr280_peak = snr280_list[peak_idx]
    snr260_peak = snr260_list[peak_idx]

    confidence = 'High' if snr280_peak >= 5.0 else 'Low'

    peak_results.append({
        'run_no': run_no,
        'Fraction_number': frac_run[peak_idx],
        'Peak_Volume_ml': peak_vol,
        'Peak_Area_280': area280,
        'Peak_Area_260': area260,
        'S/N_Ratio_280': snr280_peak,
        'S/N_Ratio_260': snr260_peak,
        'Confidence_Level': confidence
    })

# Create output dataframe
output_df = pd.DataFrame(peak_results)

# Sort by run_no, Peak_Volume_ml
output_df = output_df.sort_values(['run_no', 'Peak_Volume_ml']).reset_index(
    drop=True)

# Aggregate peak counts per run
peak_counts = output_df.groupby('run_no').size().reset_index(name='Peak_Count'
)

# Generate final markdown report using vectorized string formatting
final_output = []
final_output.append("# Peak Detection Results\n")
final_output.append("| run_no | Fraction_number | Peak_Volume_ml |
    Peak_Area_280 | Peak_Area_260 | S/N_Ratio_280 | S/N_Ratio_260 |
    Confidence_Level |")

```

```

final_output.append("
|-----|-----|-----|-----|-----|
")

# Vectorized formatting for rows
for _, row in output_df.iterrows():
    final_output.append(f"|_{row['run_no']}|_{row['Fraction_number']}|_{row[
        'Peak_Volume_ml']:.2f}|_{row['Peak_Area_280']:.2f}|_{row[
        'Peak_Area_260']:.2f}|_{row['S/N_Ratio_280']:.2f}|_{row['S/N_Ratio_260
        ']:.2f}|_{row['Confidence_Level']}|")

final_output.append("\n##_Summary\n")
final_output.append("|_run_no_|_Peak_Count_|_Excluded_Rows_(S/N<3.0)|")
final_output.append("|-----|-----|-----|")
for _, row in peak_counts.iterrows():
    final_output.append(f"|_{row['run_no']}|_{int(row['Peak_Count'])}|_{
        excluded_rows_count}|")

# Save to workspace
output_text = "\n".join(final_output)
with open('/workspace/peak_detection_results.md', 'w') as f:
    f.write(output_text)

print("Results saved to /workspace/peak_detection_results.md")
###

```

Listing 1: Code_2

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import os

# Determine the correct path for chromatography_combined.csv
# Check common locations
possible_paths = [
    '/data/chromatography_combined.csv',
    '/inputs/chromatography_combined.csv',
    os.path.join(os.getcwd(), 'chromatography_combined.csv'),
    os.path.join(os.getcwd(), 'data', 'chromatography_combined.csv'),
    os.path.join(os.getcwd(), 'inputs', 'chromatography_combined.csv')
]

# Find the file
found_path = None
for path in possible_paths:
    if os.path.exists(path):
        found_path = path
        break

if found_path is None:
    print("Error: The input file 'chromatography_combined.csv' was not found in expected locations.")
    print("Please verify the file path or ensure the file exists before running the script.")
    print(f"Current working directory: {os.getcwd()}")
    print("Contents of root directory:")
    print(os.listdir('/'))
    print("Contents of /data:")

```

```

try:
    print(os.listdir('/data'))
except FileNotFoundError:
    print("/data_directory_does_not_exist")
print("Contents of /inputs:")
try:
    print(os.listdir('/inputs'))
except FileNotFoundError:
    print("/inputs_directory_does_not_exist")
exit(1)

# Load data
df = pd.read_csv(found_path)

# Debug: Print actual column names to identify discrepancies
print("Actual_columns_in_loaded_file:", df.columns.tolist())

# Define mapping from expected names to actual names
column_mapping = {
    'Peak_Volume_ml': 'volume_ml',
    'Peak_Area_280': 'UV_1_280_ml',
    'Peak_Area_260': 'UV_2_260_ml',
    'run_no': 'run_no',
    'chromatography_stage': 'chromatography_stage'
}

# Rename columns based on the actual file structure
df = df.rename(columns=column_mapping)

# Filter relevant columns based on actual column names found above
df = df[['volume_ml', 'UV_1_280_ml', 'UV_2_260_ml', 'run_no', 'chromatography_stage']]

# Step 2: Calculate 260/280 ratio, handling zero/near-zero denominators
# Apply pd.to_numeric to ensure safe handling of potential non-numeric strings
# before replacement
df['UV_1_280_ml'] = pd.to_numeric(df['UV_1_280_ml'], errors='coerce').replace(0, 1)
df['UV_2_260_ml'] = pd.to_numeric(df['UV_2_260_ml'], errors='coerce')
df['Ratio_260_280'] = df['UV_2_260_ml'] / df['UV_1_280_ml']

# Step 3: Plot S/N distribution (using Peak_Area_280 as proxy for S/N)
plt.figure(figsize=(8, 6))
sns.boxplot(data=df, x='chromatography_stage', y='UV_1_280_ml', width=0.6)
plt.title('S/N(Peak_Area_280)_Distribution_by_Chromatography_Stage')
plt.xlabel('Chromatography_Stage')
plt.ylabel('Peak_Area_280_Proxy_for_S/N')
plt.tight_layout()
plt.savefig('/workspace/SN_distribution_by_stage.png')
plt.close()

# Step 4: Plot Area correlation (Area 260 vs Area 280)
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='UV_1_280_ml', y='UV_2_260_ml', hue='chromatography_stage', s=10, alpha=0.6)
plt.title('Area_260_vs_Area_280_Correlation_by_Stage')
plt.xlabel('Peak_Area_280')
plt.ylabel('Peak_Area_260')
plt.legend(title='Chromatography_Stage')

```

```

plt.tight_layout()
plt.savefig('/workspace/Area_correlation_by_stage.png')
plt.close()

# Step 5: Generate QC Report Text
qc_report_lines = []

# Calculate Avg S/N (Area_280) per stage
avg_sn_by_stage = df.groupby('chromatography_stage')['UV_1_280_m1'].mean().
    reset_index()
avg_sn_by_stage.columns = ['Stage', 'Avg_SN']

# List stages with >10% S/N < 4.0
stages_low_sn = df[df['UV_1_280_m1'] < 4.0].groupby('chromatography_stage').size()
stages_total = df.groupby('chromatography_stage').size()
low_sn_pct = (stages_low_sn / stages_total) * 100
stages_bad = low_sn_pct[low_sn_pct > 10].index.tolist()

# Calculate Avg Area Ratio (260/280) per stage
avg_ratio_by_stage = df.groupby('chromatography_stage')['Ratio_260_280'].mean().
    reset_index()
avg_ratio_by_stage.columns = ['Stage', 'Avg_Area_Ratio']

qc_report_lines.append("QC Report")
qc_report_lines.append("=" * 40)
qc_report_lines.append(f"Total Samples: {len(df)}")
qc_report_lines.append("")
qc_report_lines.append("Average S/N (Peak_Area_280) by Stage:")
qc_report_lines.append(avg_sn_by_stage.to_string(index=False, max_rows=None))
qc_report_lines.append("")
qc_report_lines.append("Stages with >10% Samples having S/N < 4.0:")
if stages_bad:
    qc_report_lines.append("\n".join([f"_{stage}" for stage in stages_bad]))
else:
    qc_report_lines.append("_None")
qc_report_lines.append("")
qc_report_lines.append("Average Area Ratio (260/280) by Stage:")
qc_report_lines.append(avg_ratio_by_stage.to_string(index=False, max_rows=None))

# Limit to 20 lines without arbitrary truncation of data content
if len(qc_report_lines) > 20:
    # Remove the header and footer lines to fit content within 20 lines
    # Keep header, content, and footer
    header_lines = qc_report_lines[:5]
    footer_lines = qc_report_lines[-1:] # Just the empty line at end or last line
    content_lines = qc_report_lines[5:-1]
    # Trim content if needed
    while len(header_lines) + len(content_lines) + len(footer_lines) > 20:
        content_lines.pop(0)
    qc_report_lines = header_lines + content_lines + footer_lines

qc_report_text = "\n".join(qc_report_lines)
print(qc_report_text)

```

Listing 2: Code_3